Fundamentos de Reservoir Computing

Anderson Fernandes Pereira dos Santos

Instituto Militar de Engenharia (IME), Rio de Janeiro, Brazil
Venturus Innovation Center, Campinas, Brazil
anderson@ime.eb.br | anderson.santos@venturus.org.br

Resumo. *Este artigo apresenta Reservoir Computing (RC) como o primeiro grande degrau teórico da série. O foco não é apenas definir o termo, mas mostrar como um reservatório transforma uma sequência de entradas em uma representação dinâmica rica e por que isso e útil em previsão temporal. Partimos da intuição de memória distribuída, formalizamos o modelo mínimo com equações, conectamos cada elemento do formalismo a arquivos reais do projeto e mostramos artefatos computacionais intermediários do caso Favorita. Ao final, o leitor consegue ler o código do RC clássico do projeto, entende o papel do* `washout`*, sabe o que é treinado e o que não é treinado e fica preparado para executar o primeiro pipeline completo no artigo seguinte.*

1. O que o leitor vai aprender

Ao final deste artigo, você será capaz de:

1. definir um reservatório clássico de forma matemática;
2. distinguir estado interno, vetor de entrada e readout;
3. explicar o papel de `spectral_radius`, `leak_rate` e `washout`;
4. mapear as equações de RC para a implementação em `code/rc/model.py`;
5. interpretar artefatos intermediários, e não apenas métricas finais.

2. A intuição: o que um reservatório faz, antes da equação

Em um modelo tabular tradicional, as features costumam ser calculadas explicitamente: lags, médias moveis, indicadores de calendário e assim por diante. Em RC, a ideia e diferente. Em vez de escrever manualmente toda a representação temporal, injetamos a sequência em um sistema dinamico recorrente e usamos seus estados internos como uma base de representação.

A melhor imagem mental para um iniciante e esta:

- a entrada u_t entra no sistema no instante atual;
- o sistema ainda guarda parte do que aconteceu antes;
- essa mistura entre presente e passado gera um novo estado interno;
- a previsão final e feita a partir desse estado.

Em outras palavras, o reservatório não tenta prever diretamente. Primeiro ele **transforma a sequência em estados internos**. Só depois o readout aprende a ler esses estados.

Se quisermos resumir RC em uma frase curta, ela seria:

o reservatório e um mecanismo fixo que converte sequências em representações dinâmicas; o readout e a parte treinada que converte essas representações em previsão.

Essa divisão e importante porque reduz a carga conceitual:

1. primeiro entendemos como o estado interno evolui;
2. depois entendemos como a previsão sai desse estado.

3. Construindo o modelo mínimo de RC

Em muitos textos de RC, a equação do reservatório aparece pronta, em uma unica linha. Para quem esta vendo o tema pela primeira vez, isso costuma ser brusco demais. Aqui vamos fazer o caminho inverso: começamos com a pergunta intuitiva, descemos até um unico passo de atualizacao, olhamos um exemplo de uma unica unidade e só entao escrevemos a equação completa.

3.1. A pergunta central em um unico passo

Em cada instante t, o reservatório recebe uma entrada u_t e mantem internamente um estado x_{t-1} vindo do passo anterior. O problema do passo atual pode ser formulado assim:

> dado o que entrou agora e dado o que o sistema ainda guarda do passado,
> como produzir o novo estado x_t?

Essa pergunta já organiza quase todo o raciocinio. O novo estado precisa depender de duas coisas:

- da entrada atual;
- da memória que o proprio sistema traz do passo anterior.

3.2. Os ingredientes da atualizacao

Para responder a essa pergunta, o modelo usa:

- $u_t \in \mathbb{R}^d$, o vetor de entrada no tempo t;
- $x_{t-1} \in \mathbb{R}^N$, o estado interno do passo anterior;
- W_{in}, a matriz que projeta a entrada para o espaco do reservatório;
- W_{res}, a matriz recorrente interna, que mistura os componentes do proprio estado;
- b, um vetor de bias;
- $\alpha \in (0, 1]$, o `leak_rate`, que controlara quanto do estado novo realmente entra.

O ponto principal aqui não é decorar a notação. O ponto principal e perceber o papel de cada termo:

- $W_{in}u_t$ traz o **efeito da entrada atual**;
- $W_{res}x_{t-1}$ traz o **efeito do passado interno**;
- b apenas desloca essa resposta;
- α vai decidir quanto do estado novo substitui o antigo.

3.3. Primeiro passo: montar a resposta bruta do sistema

Antes de falar em "estado candidato", vale pensar no que acontece imediatamente quando somamos os efeitos do presente e do passado. Chamaremos essa soma de pre-ativacao:

$$z_t = W_{res}x_{t-1} + W_{in}u_t + b.$$

Essa expressao pode ser lida em voz alta assim:

resposta bruta no tempo t = efeito da memória anterior + efeito da entrada atual + bias.

O vetor z_t ainda não é o novo estado final. Ele e apenas a combinacao linear inicial das influencias que atuam sobre o reservatório naquele instante.

3.4. Segundo passo: aplicar a não linearidade

Se usassemos diretamente z_t como novo estado, o modelo seria linear demais e perderia boa parte da riqueza que queremos em um reservatório. Por isso, aplicamos uma não linearidade componente a componente:

$$\widetilde{x}_t = \tanh(z_t).$$

Substituindo z_t, obtemos:

$$\widetilde{x}_t = \tanh(W_{res}x_{t-1} + W_{in}u_t + b).$$

O vetor $\widetilde{x}_t$ recebe o nome de **estado candidato**. Ele representa a resposta que o reservatório produziria se reagisse imediatamente ao passo atual.

A função $\tanh(\cdot)$ tem três papeis didaticamente importantes:

- introduz não linearidade;
- comprime os valores no intervalo $(-1, 1)$;
- ajuda a manter a dinâmica interna em uma faixa controlada.

Uma maneira simples de sentir isso e olhar alguns valores:

- se $z = 0.2$, entao $\tanh(0.2) \approx 0.197$;
- se $z = 3$, entao $\tanh(3) \approx 0.995$;
- se $z = -4$, entao $\tanh(-4) \approx -0.999$.

Isso mostra que a não linearidade não apenas "entorta"a resposta: ela também evita que os valores crescam sem limite.

3.5. Um exemplo com uma unica unidade

Para tornar a ideia menos abstrata, imagine por um momento um reservatório com apenas uma unidade. Nesse caso, todos os vetores e matrizes viram escalares:

$$z_t = w_{res}x_{t-1} + w_{in}u_t + b.$$

Suponha:

- $x_{t-1} = 0.40$;
- $u_t = 0.70$;
- $w_{res} = 0.50$;
- $w_{in} = 1.20$;
- $b = -0.10$.

Entao:

$$z_t = 0.50 \cdot 0.40 + 1.20 \cdot 0.70 - 0.10 = 0.94.$$

Aplicando a não linearidade:

$$\widetilde{x}_t = \tanh(0.94) \approx 0.735.$$

Essa conta simples ajuda a visualizar o processo:

1. o passado contribui com uma parte;
2. a entrada atual contribui com outra parte;
3. tudo isso passa pela `tanh`;
4. o resultado se torna o estado candidato.

3.6. Terceiro passo: misturar memória e atualizacao nova

O reservatório ainda não adota $\widetilde{x}_t$ como estado final. Em vez disso, ele mistura parte da memória anterior com parte do estado candidato:

$$x_t = (1 - \alpha)x_{t-1} + \alpha\widetilde{x}_t.$$

Essa e a parte que transforma o reservatório em um sistema com memória controlada. O parametro α regula a velocidade da atualizacao:

- se α e pequeno, o sistema preserva mais memória e muda devagar;
- se α e grande, o sistema reage mais ao presente e muda mais rapido.

Voltando ao exemplo anterior, se $x_{t-1} = 0.40$, $\widetilde{x}_t \approx 0.735$ e $\alpha = 0.20$, entao:

$$x_t = 0.80 \cdot 0.40 + 0.20 \cdot 0.735 \approx 0.467.$$

O leitor deve notar o seguinte: o estado final x_t ainda carrega parte do passado. Ele não salta diretamente para o estado candidato.

3.7. A equação completa do reservatório

Agora sim podemos juntar os passos anteriores. Substituindo o estado candidato na equação de mistura, chegamos a dinâmica completa:

$$x_t = (1 - \alpha)x_{t-1} + \alpha \tanh(W_{res}x_{t-1} + W_{in}u_t + b).$$

A leitura guiada dessa expressao e:

- olhe para o passado: x_{t-1};
- combine esse passado com a entrada atual: $W_{res}x_{t-1} + W_{in}u_t + b$;
- aplique a não linearidade: $\tanh(\cdot)$;
- misture o resultado com a memória anterior usando α.

Essa leitura e mais importante do que decorar a formula.

3.8. Como isso aparece no código

No código, essa construcao aparece de forma quase literal em `EchoStateReservoir.step()`:

```
pre_activation = (
    self.recurrent_weights @ self.state
    + self.input_weights @ input_vector
    + self.bias
)
new_state = np.tanh(pre_activation)
self.state = (1.0 - self.config.leak_rate) * self.state + self.config.l
```

A correspondencia entre teoria e implementação fica mais fácil de ler quando usamos os mesmos três passos:

1. `pre_activation` corresponde a z_t;
2. `new_state` corresponde a $\widetilde{x}_t$;
3. `self.state = ...` corresponde a atualizacao final de x_t.

3.9. Como a previsão sai do estado interno

A dinâmica do reservatório produz uma representação temporal, mas ainda não produz a previsão final. Para isso, usamos um readout linear:

$$\hat{y}_t = W_{out}\phi_t, \qquad \phi_t = [1; u_t; x_t].$$

O vetor ϕ_t concatena três elementos:

- um termo constante, representado por `1`;
- a entrada atual u_t;
- o estado interno x_t.

Em linguagem simples, isso quer dizer: depois que o reservatório transforma a sequência em um estado interno, o readout aprende uma combinacao linear dessas informações para produzir a previsão.

No projeto, `\phi_t` e construída por `_feature_vector()` e `W_out` e ajustado via regressão Ridge.

4. O que é treinado e o que não é treinado

Um dos pontos mais didáticos de RC e a divisão entre dinâmica fixa e leitura treinável.

No projeto:

- `W_in`, `W_res` e `b` sao inicializados aleatoriamente e mantidos fixos;
- apenas o readout linear e ajustado com supervisão.

Formalmente, o treinamento resolve

$$W_{out}^* = \arg\min_{W_{out}} \|Y - \Phi W_{out}\|_2^2 + \lambda \|W_{out}\|_2^2,$$

em que Φ é a matriz de features acumuladas ao longo do tempo e λ é o hiperparametro de regularização Ridge.

Essa escolha reduz o custo de treinamento em relação a RNNs totalmente treináveis e, ao mesmo tempo, preserva uma representação temporal rica.

5. Como o tempo entra no modelo

No projeto, o vetor de entrada sequencial e construído em `code/common/sequential_features.py` com sete componentes:

- demanda prévia normalizada;
- promoção normalizada;
- `dow_sin`, `dow_cos`;
- `month_sin`, `month_cos`;
- indicador de fim de semana.

Em notação compacta,

$$u_t = \begin{bmatrix} \widetilde{y}_{t-1} & \widetilde{p}_t & \sin(2\pi\,\mathrm{dow}_t/7) & \cos(2\pi\,\mathrm{dow}_t/7) & \sin(2\pi\,\mathrm{month}_t/12) & \cos(2\pi\,\mathrm{month}_t/12) & \mathbb{1}_{\mathrm{weekend}} \end{bmatrix}^\top$$

Esse vetor coloca em uma mesma representação:

- memória de curtissimo prazo, via $\widetilde{y}_{t-1}$;
- contexto exógeno, via promoção;
- ciclo temporal, via senos e cossenos.

6. O papel do washout

Nos primeiros instantes da simulação, o estado do reservatório ainda esta fortemente contaminado pela inicialização. Por isso, o projeto descarta os primeiros passos antes de treinar o readout.

Se chamarmos o `washout` de w, o treino do readout usa apenas os tempos $t \geq w$:

$$\Phi = [\phi_w, \phi_{w+1}, \ldots, \phi_T]^\top.$$

Em `fit_rc_readout()`, isso aparece no teste `if idx >= config.washout`.

O `washout` não é um detalhe de implementação. Ele e uma parte conceitual do pipeline, porque separa a transiente inicial da dinâmica que realmente queremos aproveitar.

7. Como o projeto implementa RC no caso Favorita

A tabela abaixo resume o pipeline implementado.

Objeto	Dimensão no projeto	Origem
Vetor de entrada u_t	7	`scaled_input_vector()`
Estado do reservatório x_t	80	`RCConfig.n_reservoir`
Vetor de features $[1; u_t; x_t]$	88	`_feature_vector()`
Readout	1 saida	Ridge linear sobre as features

O fluxo completo do código e o seguinte:

1. `build_scaler_state()` calcula médias e desvios para normalizacao;
2. `scaled_input_vector()` constroi u_t;
3. `EchoStateReservoir.step()` atualiza x_t;
4. `_feature_vector()` monta $\phi_t = [1; u_t; x_t]$;
5. `Ridge.fit()` estima o readout;
6. `forecast_rc()` faz previsão recursiva no bloco de teste.

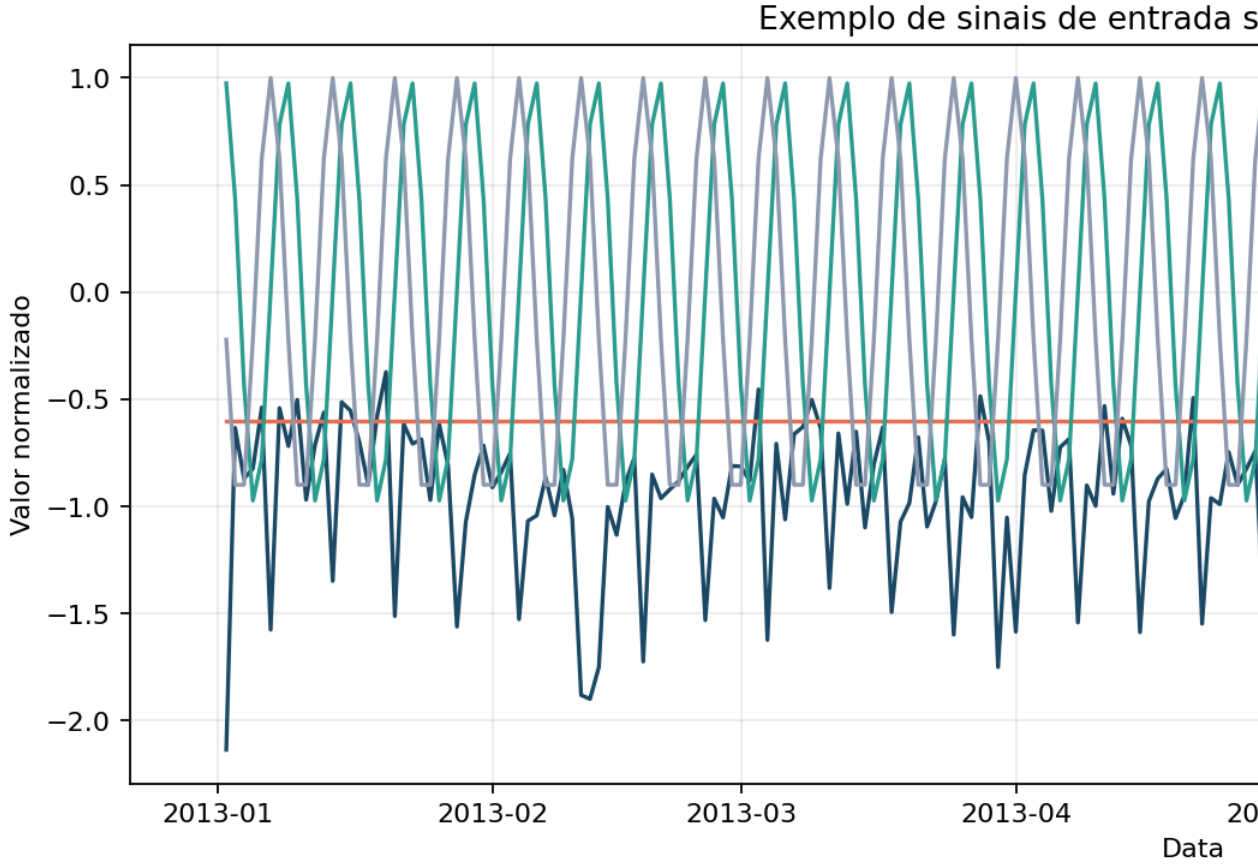
O trecho central do treinamento e este:

```
input_vector = scaled_input_vector(prev_y, row, scaler=scaler)
state = reservoir.step(input_vector)
feature_rows.append(_feature_vector(state, input_vector))
targets.append(float(row["y"]))
```

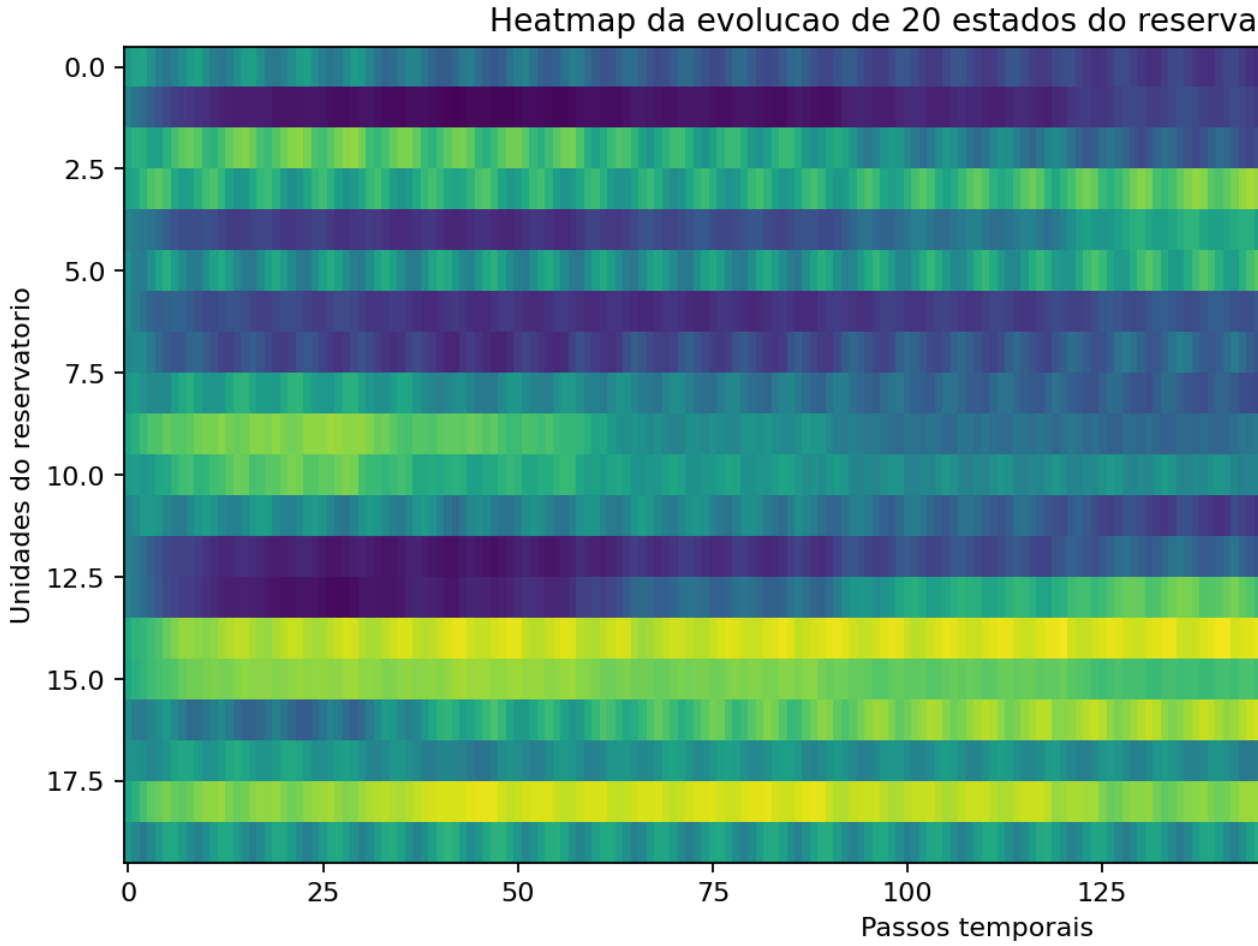
Isso mostra a ideia central de RC em sua forma mais simples: o modelo não aprende uma representação temporal diretamente por backpropagation; ele coleta estados dinamicos e aprende apenas como le-los.

8. O que os artefatos intermediários mostram

O artigo não precisa esperar a métrica final para ensinar alguma coisa. Os artefatos computacionais deste modulo mostram o pipeline em um nivel mais interno.



A imagem de entrada sequencial mostra que o reservatório não recebe apenas uma série de vendas crua. Ele recebe uma codificacao temporal e exógena compacta.



O heatmap dos estados ilustra duas ideias importantes:

1. unidades diferentes respondem de modos diferentes ao mesmo estimulo;
2. a memória do reservatório não é um unico lag explícito, mas uma combinacao distribuída de estados.

Em outras palavras, RC constroi uma base dinâmica de features sem precisar enumerar manualmente toda a interação temporal.

9. Comparando RC com outras famílias de modelo

Vale situar RC entre outros paradigmas que também aparecem na série:

- `Seasonal Naive`, `ETS` e `Prophet` modelam estrutura temporal de forma mais explícita;
- `XGBoost` usa engenharia de atributos tabulares;
- `LSTM` aprende dinâmica recorrente via gradiente;
- `RC` fixa a dinâmica recorrente e treina apenas o readout.

Essa posicao intermediária e pedagogicamente muito poderosa. RC é mais rico do que um baseline ingenuo, mas muito mais simples de treinar do que uma RNN completa.

10. Passo-a-passo para estudar ou modificar o RC do projeto

Um leitor que queira estudar a implementação deve seguir esta ordem:

1. abrir `code/common/sequential_features.py`;
2. verificar como o vetor `u_t` e construído;
3. abrir `code/rc/model.py` e localizar `RCConfig`;
4. ler `EchoStateReservoir.step()` junto da equação do artigo;
5. ler `fit_rc_readout()` e localizar o `washout`;
6. executar `python code/rc/run.py`;
7. validar o comportamento com `pytest code/rc/test_model.py`.

Essa ordem importa. Se o leitor abrir apenas `run.py`, perde a correspondencia entre teoria e implementação.

11. Conclusão

RC já esta completamente formulado no contexto do projeto. Temos uma definicao matemática, uma implementação concreta e artefatos intermediários que mostram a representação temporal em acao. O próximo passo natural e colocar esse modelo em confronto com um baseline forte, medindo o que ele realmente entrega no recorte adotado no Favorita.

Entregaveis associados no repositorio

- implementação do RC: `code/rc/model.py`
- execucao do RC: `code/rc/run.py`
- testes do RC: `code/rc/test_model.py`
- construcao de entrada sequencial: `code/common/sequential_features.py`
- artefatos deste artigo: `computational_results_20260402_222902/`

12. Referências

Referências

[1] JAEGER, H. **The "echo state" approach to analysing and training recurrent neural networks**. Bonn: German National Research Center for Information Technology (GMD), 2001.

[2] LUKOSEVICIUS, M.; JAEGER, H. **Reservoir computing approaches to recurrent neural network training**. *Computer Science Review*, v. 3, n. 3, p. 127–149, 2009.

[3] LUKOSEVICIUS, M. **A practical guide to applying echo state networks**. In: MONTAVON, G.; ORR, G. B.; MÜLLER, K.-R. (org.). *Neural Networks: Tricks of the Trade*. 2. ed. Berlin: Springer, 2012. p. 659–686.

[4] FUJII, K.; NAKAJIMA, K. **Harnessing disordered-ensemble quantum dynamics for machine learning**. *Physical Review Applied*, v. 8, n. 2, 024030, 2017.